

CSCE 625 ARTIFICIAL INTELLIGENCE · MODULE 08 · LECTURE NOTES

Logic and Satisfiability

The most useful tool nobody mentions.

MODULE OUTCOMES

- Express a problem in propositional logic.
- Explain resolution and why it is complete.
- Explain what a modern SAT solver does and why it is fast.
- Encode a real problem into CNF and solve it.
- State what first-order logic adds and what it costs.

1 Propositional logic and CNF

EVERYTHING REDUCES TO CONJUNCTIVE NORMAL FORM

A CNF formula is a conjunction of clauses, each clause a disjunction of literals (a variable or its negation). Every propositional formula has a logically equivalent CNF form.

So a solver needs to handle only one syntactic form, which is why SAT solvers accept CNF and nothing else — a deliberate simplification that pushes the conversion work onto the user and lets the solver be extremely well optimised for one case.

Naïve conversion can blow up exponentially (distributing a disjunction over a large conjunction), so the Tseitin transformation introduces auxiliary variables naming each subformula, producing a linear-size formula that is satisfiability-equivalent rather than logically equivalent.

That distinction matters. Tseitin preserves whether a model exists, not the set of models — the auxiliary variables are determined by the originals, so a satisfying assignment restricts correctly, but model counting over the Tseitin form is not the same as over the original. Which bites exactly when you want 'exactly one solution' (§3) and is worth knowing before you discover it.

What you want	Clauses	Note
At least one of $x_1 \dots x_n$	$(x_1 \vee x_2 \vee \dots \vee x_n)$	One clause. Trivial.
At most one of $x_1 \dots x_n$	$(\neg x_i \vee \neg x_j)$ for every pair.	$O(n^2)$ clauses. Use a sequential (ladder) encoding with auxiliary variables for large n , which is $O(n)$ — see the footnote.

What you want	Clauses	Note
Exactly one	Both of the above conjoined.	The workhorse for 'this variable takes exactly one value', which is how a CSP (Module 06) is encoded into SAT.
Implication $a \rightarrow b$	$(\neg a \vee b)$	One clause. All of propositional reasoning about rules reduces to this shape.
Cardinality: at most k of n	A sequential counter encoding with auxiliary variables.	$O(nk)$ clauses. Do not expand it as all $(k+1)$ -subsets, which is what a naive implementation does and is immediately hopeless.

The **at-most-one encoding** is where naive CNF generation goes wrong, and it is worth internalising because it recurs in every encoding: **pairwise is $O(n^2)$ clauses while a sequential encoding is $O(n)$** , and at n in the thousands the difference decides whether the solver sees the problem at all. **Encoding skill is what separates usable SAT from unusable SAT**, and it is the part that is learned by doing rather than by reading.

2 How solvers became fast

DPLL (1962): backtracking search plus UNIT PROPAGATION -- if a clause has exactly one unassigned literal and all the others are false, that literal MUST be true. Propagate it, and let the consequences cascade.

(This is Module 06's arc consistency, specialised to clauses -- the same inference, the same cascade.)

CDCL adds the thing that changed everything:

WHEN A CONFLICT OCCURS, ANALYSE WHY IT OCCURRED.

Walk the implication graph backwards to identify the set of decisions responsible, and LEARN A NEW CLAUSE forbidding exactly that combination. Add it to the formula permanently. Then BACKJUMP to the decision level that actually caused the conflict -- not merely one level up, which may be irrelevant.

WHY THIS IS SO POWERFUL

the learned clause prunes EVERYWHERE in the remaining search, not only here -- it is a derived fact about the formula, valid globally
backjumping skips entire irrelevant subtrees, which is Module 06's conflict-directed backjumping
and VSIDS activity-based variable ordering focuses on variables appearing in recent conflicts, so the ordering ADAPTS to the formula's structure during the search rather than being fixed in advance

THE RESULT: solvers handle formulas with MILLIONS of variables routinely. SAT is NP-complete and SAT solvers are industrial infrastructure -- exactly the situation of CSCE 669 Module 09's integer programming, for exactly the same reasons.

Learning from failure is the idea, and it is worth stating in those terms: **a conflict is not merely a dead end to retreat from, but information about the formula that can be generalised and reused. That reframing is the whole advance**, and it generalises beyond SAT — the same principle appears in CSCE 669 Module 09's cutting planes, where a violated constraint becomes a globally valid inequality rather than a local rejection.

RESOLUTION, AND WHY COMPLETENESS MATTERS

From $(a \vee X)$ and $(\neg a \vee Y)$, infer $(X \vee Y)$. That single inference rule is resolution, and it is the whole proof system for propositional logic.

It is refutation-complete: if a CNF formula is unsatisfiable, repeated application of resolution will derive the empty clause — so a finite proof of unsatisfiability always exists, even though finding it may take exponential time.

And CDCL's learned clauses *are* resolution derivations — conflict analysis is a sequence of resolutions along the implication graph. **Which is why a CDCL solver can emit a machine-checkable proof of unsatisfiability** (a DRAT proof), verified independently of the solver.

And that is the practically important part. 'Unsatisfiable' becomes a checkable claim rather than a trusted one — which is why SAT is usable in hardware verification, where 'the solver says there is no bug' must not depend on the solver being correct. **It is the same discipline this program has insisted on throughout: a result with a certificate beats a result with a reputation.**

3 What people actually solve with SAT

- **Hardware and software verification. The largest real application by a wide margin** — bounded model checking unrolls a system's transitions k steps and asks whether a bad state is reachable, which is a SAT query. Every major chip is verified partly this way.
- **Package dependency resolution. Your package manager is running a SAT solver** — 'find a set of package versions satisfying every dependency and conflict' is literally a satisfiability problem, and the switch to real solvers is why modern package managers resolve where older ones gave up.
- **Scheduling and configuration** — though **a constraint-programming or integer-programming solver is frequently better when there are costs to minimise** rather than only feasibility to establish (CSCE 669 Module 13 §1), since SAT has no native notion of an objective.
- **Puzzle solving and generation**, including **'does this generated level have exactly one solution?'** — asked by solving, forbidding the solution found, and solving again (and note §1's warning about counting over a Tseitin encoding).
- **And SMT extends it** — satisfiability modulo theories, which is SAT plus decision procedures for arithmetic, arrays, bit-vectors, and strings. **This is what program verifiers, symbolic execution engines, and type checkers for dependent types actually use**, and Z3 is free.
- **So reach for a SAT or SMT solver before writing a custom search** for anything whose core is a combinatorial feasibility question. **The solver embodies decades of work that you will not reproduce**, which is CSCE 669 Module 13's argument in a different setting.

4 First-order logic, and where logic sits

FIRST-ORDER LOGIC BUYS GENERALITY AND LOSES DECIDABILITY

Propositional logic has no variables ranging over objects, so you must enumerate: $\text{Mortal}(\text{Socrates})$, $\text{Mortal}(\text{Plato})$, and one proposition per individual — which does not scale and does not generalise to objects you have not met.

First-order logic adds quantifiers, variables, and relations, so a single sentence covers all objects. **Enormously more compact, and it is how knowledge is naturally stated**, which is why it dominated knowledge representation.

The cost is that validity becomes undecidable. It is *semi*-decidable: **a proof will eventually be found if one exists, and if none exists the search may simply never terminate** — so you cannot distinguish 'no' from 'not yet' (CSCE 627's subject, which arrives next semester).

So practical systems restrict it. Datalog (no function symbols, bounded), description logics (the basis of ontology languages), and answer set programming each give up expressiveness to regain decidability and efficient solving — and **the choice of restriction is the engineering work, not a compromise**, exactly as the choice of a convex formulation was in CSCE 669 Module 02.

WHERE LOGIC SITS RELATIVE TO THE REST OF THE COURSE

Logic was the dominant paradigm in AI for roughly thirty years and is now a specialised tool, which is a fair description rather than a dismissal — and the specialisation is in areas where it is not merely useful but indispensable.

What survived is the solvers. SAT and SMT are industrial infrastructure running in compilers, verifiers, and package managers; **general-purpose logical knowledge bases largely are not**, despite very substantial effort.

What failed was brittleness. Real-world knowledge has exceptions — birds fly, except penguins, except injured ones, except in a sealed room — and classical logic has no graceful way to say 'usually'. The attempts (default logic, non-monotonic reasoning) are technically interesting and did not resolve it, which is **Module 09's subject and the reason probability displaced logic for reasoning about the world.**

So the honest summary is: use logic where the domain really is crisp and the question is feasibility or correctness; use probability where the domain is uncertain and the question is what to believe. Circuits and package dependencies are crisp; whether the player is hiding behind that crate is not.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Russell & Norvig — AIMA, chapters 7–9 (pseudocode free)	Propositional and first-order logic, resolution, and DPLL — the reference for §1, §2, and §4.
Marques-Silva, Lynce & Malik — Conflict-Driven Clause Learning (free chapter)	The §2 mechanism , by its originators, with the implication-graph analysis worked through.
Z3 and MiniSat documentation and tutorials (free)	The practical tools of §3. The Z3 guide is interactive and is the fastest way to become useful with SMT.
Biere, Heule, van Maaren & Walsh — Handbook of Satisfiability	The comprehensive reference , with several chapters free and the annual competition results that document §2's progress.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Convert a formula to CNF naively** and watch it blow up.

- 2 **Implement the Tseitin transformation** and confirm the size is linear.
- 3 **Encode at-most-one pairwise and sequentially** for $n = 1000$ and compare clause counts and solve times.
- 4 **Implement DPLL with unit propagation** and solve small instances.
- 5 **Add conflict analysis and clause learning** and report the reduction in decisions.
- 6 **Encode Sudoku into CNF** and compare a SAT solver against your Module 06 CSP solver.
- 7 **Encode graph colouring** and find the chromatic number by repeated SAT calls.
- 8 **Get an unsatisfiability proof** from a solver and check it with an independent checker.
- 9 **Use Z3 on a problem with arithmetic** and compare against encoding the arithmetic into pure SAT.
- 10 **Use SAT to verify a generated puzzle has exactly one solution.**

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 What is CNF and why do solvers require it?
- 2 What does Tseitin preserve, and what does it not?
- 3 Give five CNF encoding patterns and the cost of each.
- 4 What is unit propagation, and what is it in Module 06's terms?
- 5 What does CDCL add to DPLL, and why is each part powerful?
- 6 State resolution and explain refutation completeness.
- 7 Why does a checkable proof matter in practice?
- 8 Name five SAT applications.
- 9 What does first-order logic buy and cost, and what do practical systems do?